UNIVERSIDAD AUTÓNOMA DE BAJA CALIFORNIA

Facultad de Ciencias Químicas e Ingeniería

Ingeniero en computación
PROYECTO DE CARRERA
Taller No. 4

PROFESOR:
ALUMNA: Gutierrez Sanchez Valeria Karey 1282379
GRUPO 581

FECHA ENTREGA: 09 de septiembre de 2026

Taller 4. Alternativas técnicas y viabilidad

1. Introducción 3
2. Alternativa A: Arquitectura basada en Raspberry Pi Zero 2 W 3
3. Alternativa B: Arquitectura basada en ESP32-CAM 4
4. Matriz de decisión ponderada 5
5. Comparación detallada 6
6. Justificación de la alternativa elegida 7
7. Lista preliminar de materiales 8
8. Estimación preliminar de costos recurrentes de nube o comunicación 8
9. Viabilidad 9
10. Restricciones y riesgos técnicos principales 10
11. Actualización del alcance aprobado 10
12. Referencias 12

Taller 4. Alternativas técnicas y viabilidad

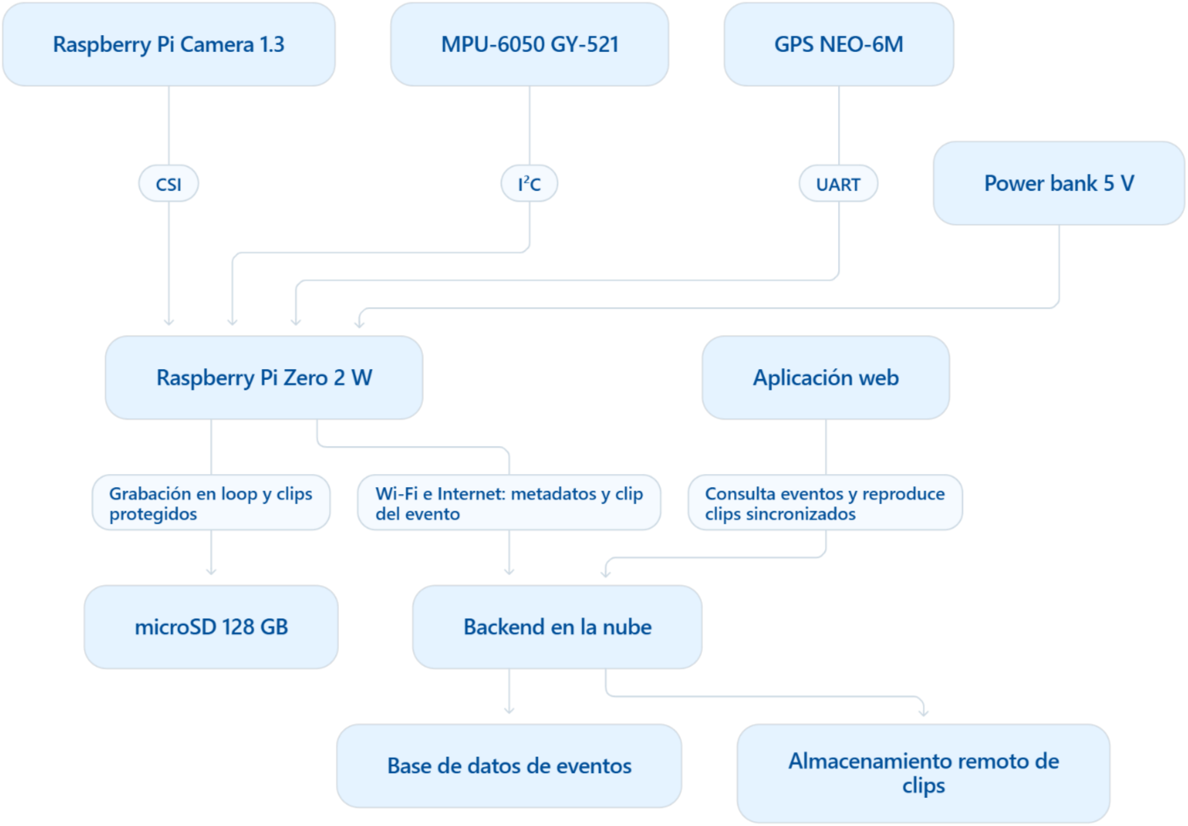
1. Introducción

En este documento se presentan dos alternativas técnicas completas para implementar DashGuard. Ambas alternativas consideran el flujo completo del sistema: adquisición de datos en el sistema embebido, almacenamiento local de video, envío de metadatos mediante Internet, backend desplegado en la nube, base de datos y una interfaz web responsiva.

2. Alternativa A: Arquitectura basada en Raspberry Pi Zero 2 W

Sistema embebido de placa única con Raspberry Pi OS. La Raspberry Pi Zero 2 W realiza la captura de video, lectura del MPU-6050 y del GPS NEO-6M, detección básica de eventos y almacenamiento local de las grabaciones en una microSD de 128 GB. Mediante Wi-Fi envía al backend en la nube los metadatos de cada evento. Los clips asociados a eventos se conservan y pueden sincronizarse con almacenamiento remoto para que el usuario los reproduzca desde la aplicación web.

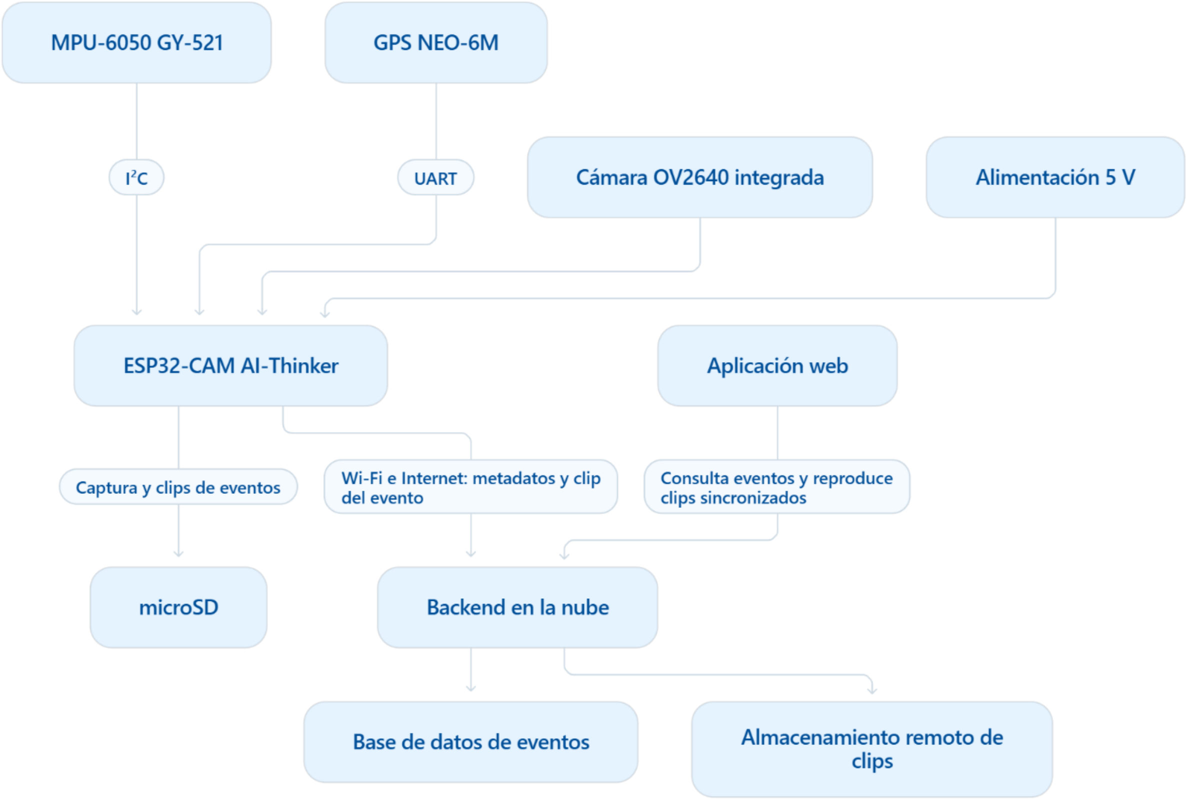
Componentes principales: Raspberry Pi Zero 2 W (SoC RP3A0, CPU ARM Cortex-A53 de 4 núcleos, 512 MB de RAM) [1]; Raspberry Pi Camera Module 1.3 mediante CSI; IMU MPU-6050 GY-521 por I2C [2]; módulo GPS NEO-6M por UART; microSD para sistema y grabaciones; conectividad Wi-Fi; y alimentación de 5 V mediante power bank. El backend se implementará como una API web desplegada en la nube, con una base de datos para metadatos y almacenamiento de objetos/archivos para los clips de eventos.


Raspberry Pi Camera 1.3
MPU-6050 GY-521
GPS NEO-6M
CSI
I^2C
UART
Power bank 5 V
Raspberry Pi Zero 2 W
Aplicación web
Grabación en loop y clips protegidos
Wi-Fi e Internet: metadatos y clip del evento
Consulta eventos y reproduce clips sincronizados
microSD 128 GB
Backend en la nube
Base de datos de eventos
Almacenamiento remoto de clips

3. Alternativa B: Arquitectura basada en ESP32-CAM

Sistema basado en ESP32-CAM AI-Thinker, con cámara OV2640 integrada y conectividad Wi-Fi. El microcontrolador realiza la captura de imágenes/video, lectura del MPU-6050 y del GPS NEO-6M y almacenamiento en microSD. Los datos de eventos se enviarían mediante HTTP/HTTPS al mismo tipo de backend en la nube utilizado por la Alternativa A. Los clips asociados a eventos también tendrían que transferirse al almacenamiento remoto para poder reproducirse desde la aplicación web.

Componentes principales: ESP32-CAM AI-Thinker (Xtensa LX6 de doble núcleo, ~520 KB de SRAM y PSRAM disponible según el módulo) [3]; cámara OV2640 integrada; MPU-6050 por I2C [2]; GPS NEO-6M por UART; microSD; Wi-Fi; backend en la nube, base de datos de metadatos y almacenamiento remoto de clips. Esta alternativa reduce costo y consumo, pero dispone de menos recursos para manejar simultáneamente captura de video, sensores, sistema de archivos y transferencia de grabaciones.


MPU-6050 GY-521
GPS NEO-6M
I²C
UART
Cámara OV2640 integrada
Alimentación 5 V
ESP32-CAM AI-Thinker
Aplicación web
Captura y clips de eventos
Wi-Fi e Internet: metadatos y clip del evento
Consulta eventos y reproduce clips sincronizados
microSD
Backend en la nube
Base de datos de eventos
Almacenamiento remoto de clips

4. Matriz de decisión ponderada

Cada alternativa se calificó en una escala de 1 a 5, donde 1 representa una condición muy desfavorable y 5 una condición muy favorable. La calificación ponderada se obtiene multiplicando la calificación de cada alternativa por el peso asignado al criterio. La suma de las calificaciones ponderadas determina el resultado final de cada alternativa.

Criterio	Peso	Raspberry Calif.	Raspberry Ponderado	ESP32 Calif.	ESP32 Ponderado
Procesamiento video + IMU + GPS	20 %	5	1.00	2	0.40
Almacenamiento local	10 %	5	0.50	3	0.30
Integración de sensores	10 %	4	0.40	3	0.30
Comunicación con backend	10 %	5	0.50	4	0.40
Costo de hardware	15 %	3	0.45	5	0.75
Consumo energético	10 %	2	0.20	5	0.50
Complejidad de desarrollo	15 %	4	0.60	2	0.30
Tiempo de implementación	10 %	4	0.40	3	0.30
TOTAL	**100 %**	—	**4.05**	—	**3.25**

La Alternativa A obtiene una puntuación ponderada de 4.05 frente a 3.15 de la Alternativa B. El ESP32-CAM presenta ventajas claras en costo y consumo energético; sin embargo, para el alcance de DashGuard la Raspberry Pi Zero 2 W ofrece mayor margen para manejar simultáneamente video, sensores, almacenamiento local, protección de clips de eventos y comunicación con servicios en la nube. Esta diferencia reduce el riesgo de integración dentro del plazo disponible de nueve semanas.

5. Comparación detallada

Aspecto	Alternativa A: Raspberry Pi Zero 2 W	Alternativa B: ESP32-CAM
Sistema embebido	SBC con Raspberry Pi OS, 512 MB de RAM y CPU de cuatro núcleos. Permite separar la captura de video, lectura de sensores, gestión de archivos y comunicación de red en procesos o servicios.	Microcontrolador con recursos considerablemente menores. Puede usar tareas de FreeRTOS, pero la captura de cámara, microSD, sensores y red compiten por memoria y tiempo de CPU.
Sensores y actuadores principales	Raspberry Pi Camera Module 1.3 por CSI, MPU-6050 GY-521 por I2C, GPS NEO-6M por UART e indicadores básicos.	OV2640 integrada, MPU-6050 por I2C, GPS NEO-6M por UART e indicadores básicos.
Comunicación	Wi-Fi 802.11 b/g/n para enviar metadatos y clips de eventos al backend mediante HTTP/HTTPS; UART e I2C para sensores.	Wi-Fi integrado para HTTP/HTTPS. UART e I2C son utilizables, aunque la disponibilidad de pines y recursos es más limitada por la cámara y microSD.
Backend y servicio en la nube	API web desplegada en un servicio de nube. Recibe y valida eventos enviados por la Raspberry, registra metadatos y gestiona la referencia al clip asociado.	Utilizaría el mismo backend en la nube; el ESP32-CAM actuaría como cliente y enviaría eventos y clips.
Base de datos	Base de datos en la nube para fecha/hora, ubicación, tipo de evento, valores del IMU y referencia del video. Los archivos de video se mantienen separados de la base de datos, en almacenamiento de objetos/archivos.	Misma base de datos y almacenamiento remoto que la Alternativa A. La diferencia está en la capacidad del dispositivo para preparar y transferir los clips.
Frontend	Aplicación web responsiva que consulta el backend, muestra los eventos registrados y permite reproducir los clips asociados disponibles en el almacenamiento remoto.	La misma aplicación web puede utilizarse. La experiencia depende de que el ESP32-CAM consiga generar y transferir correctamente los clips asociados a los eventos.
Costo	Costo inicial mayor, aunque se utilizarán los componentes ya adquiridos/cotizados para el prototipo.	Menor costo del módulo principal y cámara integrada.
Complejidad	Media-alta. Linux y Python facilitan el manejo de archivos, procesos, HTTP y bibliotecas para cámara y sensores.	Alta para este alcance: requiere administrar cuidadosamente memoria, concurrencia, cámara, microSD, sensores y transferencia de archivos.

Tiempo de desarrollo	Compatible con un MVP en aproximadamente 9 semanas si se priorizan grabación en loop, eventos, backend y reproducción de clips asociados.	Mayor riesgo temporal por la optimización necesaria para integrar video, sensores, almacenamiento y red con recursos limitados.
Seguridad	Puede usar HTTPS, autenticación/token del dispositivo, validación de la API y control de acceso en la aplicación web.	También puede usar HTTPS y tokens, pero la implementación debe ajustarse a los recursos del microcontrolador.

6. Justificación de la alternativa elegida

Se selecciona la Alternativa A (Raspberry Pi Zero 2 W) porque ofrece un margen técnico mayor para cumplir el alcance actualizado de DashGuard dentro del periodo disponible de nueve semanas:

1. **Capacidad de procesamiento y manejo de archivos.** Se debe capturar video en loop, leer IMU y GPS, detectar eventos, proteger los clips asociados y comunicarse con un backend. La Raspberry Pi Zero 2 W dispone de un entorno Linux y recursos superiores a los del ESP32-CAM para ejecutar estas funciones de manera concurrente [1]. El trabajo de Pranoto et al. también demuestra el uso de una Raspberry Pi Zero 2 W para procesamiento de señales inerciales en tiempo real [4].
2. **Gestión y acceso a las grabaciones.** Las grabaciones continuas se almacenarán localmente en una microSD de 128 GB. Cuando se detecte un evento, el segmento correspondiente se marcará para evitar que el loop lo sobrescriba y podrá sincronizarse con almacenamiento remoto. El backend guardará la referencia del archivo para que la aplicación web permita reproducir la grabación asociada al evento.
3. **Arquitectura web y seguridad.** El dispositivo enviará los eventos al backend mediante HTTP/HTTPS. La API validará los campos indispensables y podrá identificar el dispositivo mediante un token. La base de datos almacenará metadatos, mientras que los videos se manejarán como archivos separados, evitando guardar binarios grandes directamente en la base de datos.
4. **Consumo energético.** El ESP32-CAM mantiene una ventaja importante en consumo. Para el prototipo, esta desventaja de la Raspberry se acepta a cambio de reducir el riesgo de integración. La autonomía real deberá medirse con la cámara, GPS, IMU y Wi-Fi funcionando antes de fijar la batería definitiva.

La matriz ponderada (4.05 frente a 3.15) refleja esta decisión: la Raspberry Pi Zero 2 W no es la alternativa de menor costo ni de menor consumo, pero obtiene mejores resultados en los criterios más relacionados con el alcance funcional de DashGuard, especialmente procesamiento, gestión de grabaciones y complejidad de integración.

7. Lista preliminar de materiales

#	Componente	Modelo de referencia	Cantidad	Costo aproximado unitario (MXN)	Subtotal (MXN)
1	**Placa de cómputo**	Raspberry Pi Zero 2 W	1	$1300.00	$1,300.00
2	**Módulo de cámara**	Raspberry Pi Camera 1.3 (CSI)	1	$255.00	$255.00
3	**Módulo GPS**	NEO-6M con antena	1	$115.00	$115.00
4	**Sensor inercial (IMU)**	MPU-6050 GY-521	1	$132.18	$132.18
5	**Almacenamiento**	MicroSD 128 GB Class 10	1	$485.00	$485.00
6	**Alimentación**	Power bank 10,000 mAh, salida 5 V / 2 A o superior	1	$350.00	$350.00
7	**Gabinete e instalación**	Case/adaptación y soporte para parabrisas	1	$180.00	$180.00
	Total estimado				**$2,817.18 MXN**

Nota: Precios sujetos a cambio, se utilizan los precios de adquisición o cotización disponibles para los componentes principales del prototipo

8. Estimación preliminar de costos recurrentes de nube o comunicación

Si se utilizarán servicios en la nube para recibir eventos, almacenar metadatos y permitir que la aplicación web consulte la información. Para la etapa académica se priorizará un nivel gratuito o de desarrollo del proveedor seleccionado, por lo que el costo recurrente objetivo durante el prototipo es de $0 MXN/mes. Esta estimación depende de permanecer dentro de los límites gratuitos del servicio y deberá actualizarse cuando se seleccione el proveedor definitivo.

El video continuo no se enviará permanentemente a la nube. Las grabaciones se conservarán en la microSD de 128 GB mediante un esquema de loop y se priorizará la carga de clips asociados a eventos. Esta decisión reduce el consumo de ancho de banda y almacenamiento remoto y hace viable la consulta de evidencia desde la aplicación web sin intentar respaldar en Internet todas las horas de conducción.

- La comunicación del prototipo utilizará Wi-Fi disponible; no se contempla un módem celular ni un plan de datos dentro del alcance de estas nueve semanas. Si el vehículo no tiene conexión a Internet al ocurrir un evento, el sistema deberá conservar localmente el registro y el clip para sincronizarlos cuando vuelva a disponer de conectividad.
- Los costos futuros de almacenamiento y transferencia dependerán del número y duración de los clips subidos. Por ello, una versión posterior deberá definir políticas de retención, compresión y eliminación de archivos antes de estimar un costo mensual de operación comercial.

9. Viabilidad

- Viabilidad técnica: Media-alta. La arquitectura utiliza componentes ya definidos y separa responsabilidades: la Raspberry realiza adquisición, procesamiento y almacenamiento local; el backend recibe eventos; la base de datos almacena metadatos; y el almacenamiento remoto conserva únicamente los clips que deban consultarse desde la aplicación web. La principal validación técnica pendiente es medir el desempeño real de video, sensores y sincronización de archivos funcionando simultáneamente.
- Viabilidad económica: Alta para un prototipo académico. Los componentes principales ya cotizados suman $1,152.18 MXN, más la microSD de 128 GB, alimentación y montaje aún por confirmar. Se buscará utilizar niveles gratuitos de nube durante el desarrollo y limitar la carga remota a clips asociados a eventos para controlar almacenamiento y transferencia.
- Viabilidad operativa: Media-alta. El usuario podrá consultar desde una aplicación web los eventos registrados y reproducir los clips que hayan sido sincronizados. La grabación continua permanecerá en la microSD. Si no existe conexión a Internet en el vehículo, los eventos y clips deberán permanecer en cola local hasta que sea posible sincronizarlos.
- Viabilidad temporal: Media. Quedan aproximadamente nueve semanas para integrar video, MPU-6050, GPS, almacenamiento en loop, backend, base de datos, almacenamiento remoto y frontend. Para mantener el proyecto

realizable, el producto mínimo viable priorizará: grabación en loop, detección básica de eventos, ubicación, registro en la nube y reproducción web de los clips asociados. Funciones avanzadas se dejarán como ampliaciones si el tiempo lo permite.

10. Restricciones y riesgos técnicos principales

Riesgo/restricción	Impacto	Mitigación propuesta
Saturación de CPU/RAM al ejecutar video + IMU + GPS + red simultáneamente	Alto	Probar integración desde etapas tempranas, usar procesamiento ligero del IMU y evitar procesamiento de video innecesario. Medir CPU/RAM durante pruebas.
Falsos positivos/negativos en la detección de impactos o eventos	Medio-alto	Calibrar umbrales del MPU-6050 mediante pruebas controladas y registrar datos crudos para ajustar los criterios de detección [5].
Pérdida o corrupción de grabaciones por escritura continua en microSD	Medio-alto	Implementar grabación por segmentos, cierre correcto de archivos, rotación controlada y protección de los clips asociados a eventos antes de sobrescribirlos.
Falta de Internet al momento de un evento	Medio	Guardar localmente el evento y el clip en una cola de sincronización y enviarlos cuando la Raspberry recupere conectividad Wi-Fi.
Carga lenta o fallida de clips al almacenamiento remoto	Medio	Limitar duración/tamaño de los clips, reintentar transferencias fallidas y mostrar en la web el estado de sincronización del video.
Señal GPS insuficiente en estacionamientos techados o interiores	Medio	Registrar el evento aunque no exista posición válida y marcar la ubicación como no disponible; la detección por IMU seguirá funcionando.

11. Actualización del alcance aprobado

Con base en este análisis, se propone una actualización del alcance aprobado para alinear el proyecto con el requisito de acceso web a los eventos y sus grabaciones, sin convertir el prototipo en un sistema de streaming continuo:

- DashGuard deberá grabar video en loop, almacenar las grabaciones en una microSD de 128 GB, detectar eventos mediante el MPU-6050, obtener ubicación mediante el NEO-6M, enviar los datos del evento a un backend en la nube y permitir consultar desde la aplicación web los eventos y reproducir las grabaciones asociadas. Los metadatos se almacenarán en una base de datos en la nube y los clips de eventos se manejarán mediante almacenamiento remoto de archivos/objetos.

- La grabación continua completa permanecerá localmente en la microSD. Para mantener viable el proyecto en nueve semanas y reducir consumo de red y almacenamiento, se priorizará la sincronización de los clips asociados a eventos. El acceso remoto a todas las horas de grabación, el streaming en vivo y la conectividad celular permanente quedan fuera del producto mínimo viable.
- Se mantienen fuera del alcance la integración con aseguradoras, el reconocimiento automático de placas y funciones de inteligencia artificial avanzadas. La clasificación de frenados, aceleraciones o curvas podrá incorporarse si la integración del flujo principal se completa dentro del calendario disponible.

12. Referencias

[1] Raspberry Pi Ltd., *Raspberry Pi Zero 2 W Product Brief*, Raspberry Pi Ltd., Cambridge, U.K., Apr. 2024. [Online]. Available: https://datasheets.raspberrypi.com/rpizero2/raspberry-pi-zero-2-w-product-brief.pdf

[2] InvenSense Inc., *MPU-6000/MPU-6050 Product Specification*, Rev. 3.4, InvenSense Inc., Sunnyvale, CA, USA, Aug. 2013. [Online]. Available: https://www.cdiweb.com/datasheets/invensense/mpu-6050_datasheet_v3%204.pdf

[3] Ai-Thinker, *ESP32-CAM Datasheet*, LCSC Electronics. [Online]. Available: https://lcsc.com/datasheet/lcsc_datasheet_1809181822_Ai-Thinker-ESP32-CAM_C277946.pdf

[4] H. Pranoto, A. Wahab, Y. Yoppy, M. I. Sudrajat, D. Mandaris, I. Supono, A. V. Ega, T. A. W. Wijanarko, and H. W. Nugroho, "Real-Time Vehicle Emergency Braking Detection with Moving Average Method Based on Accelerometer and Gyroscope Data," *Vehicles*, vol. 7, no. 4, p. 106, Sep. 2025, doi: 10.3390/vehicles7040106.

[5] M. Garrosa and M. Ceccarelli, "A Novel IMU-Based Aggressiveness Index for Driver Behavior Assessment Using Wearable Sensing," *Machines*, vol. 14, no. 6, p. 582, May 2026, doi: 10.3390/machines14060582.